

VLSI Implementation of DPD for Next-Generation Digital TV Transmitters

Ausilon V. Souza, Tarciso Gregório B. de Bello
 ausilon.souza@mtel.inatel.br, tarciso.gregorio@mtel.inatel.br
 Instituto Nacional de Telecomunicações – INATEL

Resumo—Este artigo apresenta o desenvolvimento de uma arquitetura HDL para Pré-Distorção Digital (Digital Predistortion – DPD) aplicada a transmissores de televisão digital de próxima geração. O trabalho considera como alvo tecnológico o contexto da TV 3.0 brasileira e adota, como sinal de referência inicial, uma forma de onda OFDM compatível com a família ATSC 3.0/A/322, canal de 6 MHz, modulação 256-QAM, FFT 8K e taxa de amostragem de 24 MS/s. A metodologia integra geração de dataset em GNU Radio, estudo algorítmico em OpenDPD, validação HDL no Questa e avaliação física preliminar em SKY130 por meio do OpenLane. O algoritmo adotado é um Generalized Memory Polynomial (GMP) com 39 termos complexos, amostras em ponto fixo Q1.15 e coeficientes em Q2.16. A arquitetura separa o caminho rápido de inferência, implementado pelo *GMPengine*, do caminho lento de captura, treinamento NLMS, atualização de coeficientes e supervisão por métricas. Os resultados atuais consolidam o contrato algoritmo–HDL, demonstram validação funcional dos blocos críticos e apresentam uma primeira organização física macro-level do sistema, incluindo estimativas de área, tempo de execução de síntese e limitações ainda abertas para fechamento de chip.

Index Terms—Digital Predistortion, DPD, VLSI, Digital TV, TV 3.0, ATSC 3.0, GMP, NLMS, HDL, SKY130, OpenLane, GNU Radio, OpenDPD.

I. INTRODUÇÃO

O trabalho apresentado neste artigo propõe o desenvolvimento de um hardware dedicado para aplicações em telecomunicações, com foco específico na implementação de um sistema de Pré-Distorção Digital (Digital Predistortion – DPD). A arquitetura desenvolvida foi concebida considerando os requisitos impostos pelos sistemas de radiodifusão de televisão digital de nova geração, em especial o padrão TV 3.0, cuja adoção representa um importante avanço tecnológico ao incorporar novas técnicas de modulação, codificação e transmissão de sinais. Nesse contexto, a crescente complexidade dos algoritmos de processamento digital de sinais exige soluções computacionais capazes de operar em tempo real, com elevada eficiência energética e reduzida latência, tornando o desenvolvimento de arquiteturas dedicadas uma alternativa de grande relevância para aplicações práticas.

Uma das premissas fundamentais deste trabalho consiste na adoção de um fluxo de desenvolvimento baseado predominantemente em ferramentas, frameworks e ambientes de desenvolvimento de código aberto. Essa escolha está diretamente relacionada à necessidade de tornar o processo de pesquisa mais acessível, transparente e reproduzível. No campo da microeletrônica e do projeto de circuitos integrados, grande parte das ferramentas de projeto eletrônico (Electronic Design

Automation – EDA) é disponibilizada apenas por meio de licenças comerciais de alto custo, geralmente restritas a grandes empresas e centros de pesquisa. Essa realidade representa uma barreira significativa para universidades, laboratórios independentes e pesquisadores, limitando o acesso às tecnologias mais modernas e dificultando a formação de novos profissionais na área.

Nos últimos anos, entretanto, observa-se um crescimento expressivo do ecossistema de ferramentas abertas para projeto de hardware digital. Esse movimento tem possibilitado a realização de fluxos completos de desenvolvimento utilizando soluções livres, abrangendo desde a descrição do hardware até as etapas de síntese lógica, posicionamento físico, roteamento, geração de layout e verificação do projeto. Além de reduzir custos, essa abordagem favorece a reprodutibilidade científica, permitindo que outros pesquisadores validem, reproduzam e estendam os resultados apresentados.

No fluxo de desenvolvimento adotado neste trabalho, a descrição do hardware foi realizada utilizando linguagem HDL, empregando ferramentas da Siemens EDA disponibilizadas por meio de licença acadêmica estudantil para validação funcional. Após a descrição do hardware, as etapas de síntese lógica, planejamento físico, posicionamento, roteamento e verificação física foram avaliadas utilizando o framework OpenLane, que integra ferramentas como Yosys, OpenROAD, Magic e Netgen ao PDK aberto SKY130 [1], [2]. A modelagem comportamental do amplificador de potência e o treinamento do DPD foram conduzidos no OpenDPD [3], [4], enquanto a geração do sinal em banda-base foi feita em GNU Radio [5], a partir de uma cadeia ATSC 3.0 disponível no projeto *gr-atsc3* [6].

A integração dessas ferramentas demonstra que é possível estabelecer um fluxo completo de desenvolvimento para sistemas de processamento digital de sinais voltados às telecomunicações utilizando predominantemente tecnologias abertas. Além de reduzir custos, essa abordagem promove maior transparência metodológica, facilita a reprodutibilidade dos experimentos e incentiva a colaboração entre pesquisadores. Dessa forma, o presente trabalho busca apresentar uma arquitetura dedicada para implementação de DPD aplicada ao contexto da TV 3.0 e, simultaneamente, registrar os contratos algorítmicos, numéricos e físicos necessários para levar esse tipo de algoritmo a uma implementação VLSI.

II. OBJETIVO E CONTRIBUIÇÕES

O objetivo principal deste trabalho é desenvolver e validar uma arquitetura HDL de DPD capaz de operar como bloco dedicado em um transmissor de televisão digital de próxima geração. Nesta etapa, o foco é consolidar a transição entre o algoritmo treinado em alto nível e uma implementação sintetizável, com atenção especial à quantização, ao particionamento entre caminho rápido e caminho lento, à validação por testbenches e à estimativa física em SKY130.

As principais contribuições desta versão do trabalho são:

- definição de um fluxo de dataset para DPD baseado em GNU Radio e em forma de onda ATSC 3.0/A/322, com canal de 6 MHz e taxa de 24 MS/s;
- estudo algorítmico em OpenDPD para seleção de um modelo GMP com 39 termos complexos e avaliação por NMSE, EVM e ACLR;
- consolidação do contrato numérico em Q1.15 para amostras I/Q e Q2.16 para coeficientes complexos;
- implementação HDL dos blocos *GMPengine*, *MACcore* com treinamento block-NLMS e *MetricEngine* com métricas L1/EWMA;
- validação funcional em Questa com vetores derivados do OpenDPD e testbenches de integração;
- avaliação física preliminar em OpenLane/SKY130 com macros individuais, floorplan macro-level e tabela de resultados de área, células, DRC/LVS e frequência estimada.

III. SINAL DE REFERÊNCIA E DATASET

A especificação brasileira TV 3.0 é tratada neste trabalho como alvo tecnológico de longo prazo. Os documentos do Fórum SBTVD e os relatórios da fase 3 indicam a adoção de uma camada física compatível com a família ATSC 3.0, incluindo conceitos presentes nos documentos A/300, A/321, A/322 e A/324 [7]–[13]. A norma brasileira ABNT NBR 25601:2025 aparece como referência formal de camada física no catálogo da ABNT [14].

Para avançar antes da disponibilidade de uma cadeia completa de transmissão TV 3.0 em hardware, foi adotado um sinal OFDM de referência inspirado no perfil físico ATSC 3.0/A/322. O dataset principal foi gerado no GNU Radio a partir do projeto *gr-atsc3*, usando como base o exemplo *vv031.grc*. Esse fluxo inclui blocos de processamento típicos da cadeia física, como scrambler, codificação BCH/LDPC, interleaving, mapeamento, inserção de pilotos, bootstrap e formação OFDM. A entrada utilizada foi um arquivo de transporte MPEG-TS, e a saída foi salva como banda-base complexa.

O dataset final possui FFT 8K, modulação 256-QAM, canal de 6 MHz e taxa de saída de 24 MS/s. A taxa de 24 MS/s foi escolhida por representar quatro vezes a largura de canal de 6 MHz, oferecendo margem para observar componentes fora de banda e distorções não lineares relevantes ao DPD. Após a geração em ponto flutuante, o sinal foi convertido para arquivos usados pelo OpenDPD e para arquivos *.hex* em Q1.15 usados nos testbenches HDL. Em simulação, os arquivos de referência e feedback são mantidos separados

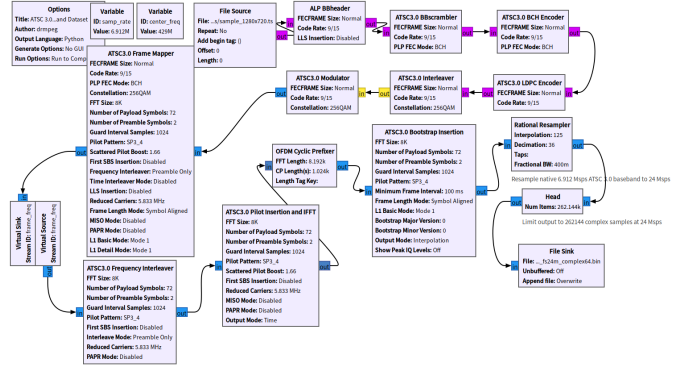


Figura 1: Fluxo GNU Radio utilizado para geração do dataset ATSC 3.0/A/322 em banda-base complexa.

para tornar o experimento determinístico; em hardware real, o feedback será produzido pela cadeia de observação após o PA.

IV. ESTUDO ALGORÍTMICO COM OPENDPD

Antes da implementação HDL, o OpenDPD foi utilizado como ambiente de referência para estudar o comportamento do PA e do predistorter. Essa etapa separa dois problemas: identificar ou aproximar a resposta do amplificador e obter uma função inversa prática capaz de compensar compressão de ganho, rotação de fase e efeitos de memória. O PA completo não é implementado no caminho rápido do chip; o que precisa operar em tempo real é o modelo DPD treinado, enquanto a atualização dos coeficientes pode ocorrer em background.

O algoritmo selecionado foi o Generalized Memory Polynomial (GMP). A escolha foi motivada por três fatores. Primeiro, o GMP é uma classe conhecida para DPD em PAs com não linearidade e memória. Segundo, sua estrutura pode ser implementada com atrasos, magnitudes, produtos complexos e acumulações, o que é mais direto para HDL do que modelos neurais densos. Terceiro, o OpenDPD permitiu treinar e exportar uma configuração GMP que pôde ser comparada contra a implementação inteira em hardware.

Na configuração consolidada, o PA foi treinado como *PA_S_0_M_GMP_H_23_F_64* e o predistorter como *DPD_S_0_M_GMP_H_15_F_64_P_39*. O parâmetro *P_39* definiu a implementação de 39 termos GMP. Como cada termo utiliza coeficiente complexo, a arquitetura passou a exigir pares real/imaginário para cada coeficiente, e não apenas coeficientes reais escalares.

O melhor ponto registrado ocorreu na época 12 de 15. A Tabela I mostra os resultados extraídos do CSV final. A diferença entre validação e teste no ACLR foi mantida como observação técnica, pois o ACLR é sensível ao trecho temporal avaliado, à ocupação espectral instantânea e à janela de medição.

V. ARQUITETURA DIGITAL PROPOSTA

A Fig. 3 apresenta a arquitetura funcional. O sistema foi estruturado em dois caminhos. O caminho rápido contém o *GMPengine*, responsável pela inferência DPD em tempo real.

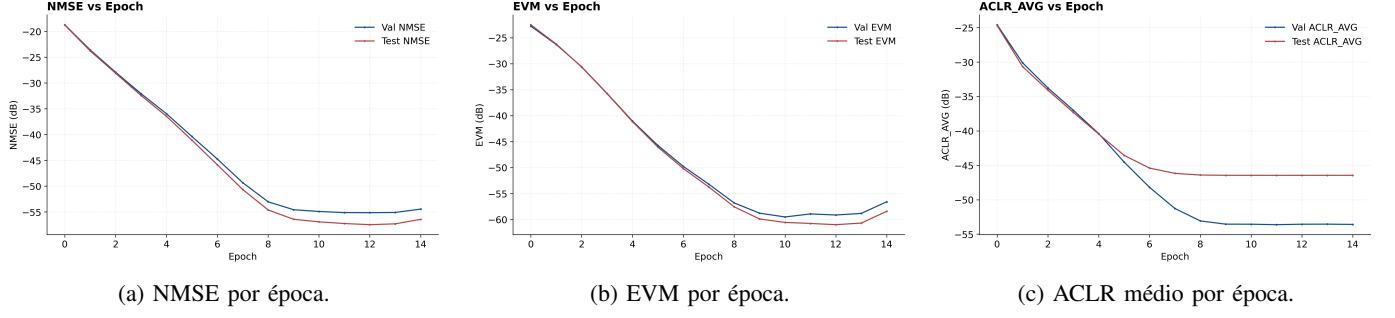


Figura 2: Curvas de treinamento obtidas no OpenDPD para o dataset ATSC 3.0/A/322 8K 256-QAM em 24 MS/s.

Tabela I: Resultados do DPD GMP no OpenDPD para o dataset ATSC 3.0/A/322.

Conjunto	NMSE (dB)	EVM (dB)	ACLR médio (dB)
Validação	-55.15	-59.13	-53.56
Teste	-57.47	-61.00	-46.45

O caminho lento contém captura de amostras, treinamento, atualização dos bancos de coeficientes e supervisão. Essa separação evita que o treinamento, que pode demandar janelas temporais muito maiores que o período de amostragem, interfira no fluxo contínuo de transmissão.

No diagrama, o bloco PA representa uma abstração do caminho de transmissão analógico/RF observado pelo sistema de DPD. Portanto, além do amplificador de potência propriamente dito, esse bloco inclui efeitos equivalentes de conversão digital-analógica, filtragem de reconstrução, upconversion, cadeia RF, acoplamento de feedback, downconversion e conversão analógico-digital, quando observados no domínio de banda-base complexa. O modelo usado pelo DPD não busca descrever cada estágio físico individualmente, mas a relação comportamental equivalente entre o sinal de referência aplicado ao transmissor e o sinal de feedback medido após a cadeia RF.

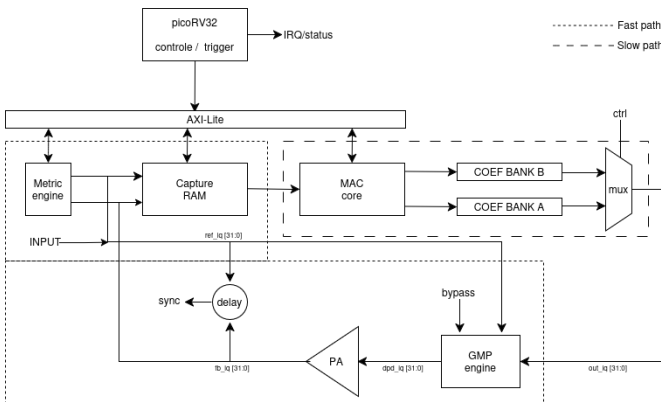


Figura 3: Diagrama funcional da arquitetura DPD com caminho rápido de inferência e caminho lento de captura/treinamento.

Durante a inicialização, o transmissor opera em bypass porque ainda não há coeficientes treinados. Nessa fase, a RAM de captura registra pares de referência e feedback. O

MACcore lê esse snapshot e executa treinamento block-NLMS sobre a mesma base GMP usada na inferência. Ao final do treinamento, os coeficientes são gravados no banco inativo. A troca de banco ocorre apenas em evento de sincronismo, evitando descontinuidade no caminho de amostras.

VI. FORMULAÇÃO MATEMÁTICA PARA HDL

As amostras complexas são representadas por

$$x[n] = x_i[n] + jx_q[n], \quad (1)$$

com componentes signed Q1.15. Os coeficientes complexos do DPD são

$$c_k = a_k + jb_k, \quad (2)$$

armazenados em signed Q2.16. Cada termo GMP ocupa dois endereços no banco de coeficientes:

$$\text{addr}(2k) = \Re\{c_k\}, \quad \text{addr}(2k+1) = \Im\{c_k\}. \quad (3)$$

A. GMPengine

O modelo implementado pelo *GMPengine* é:

$$y[n] = \sum_{k=0}^{38} c_k \phi_k[n], \quad (4)$$

em que

$$\phi_k[n] = x_{m_x(k)}[n] |x_{m_a(k)}[n]|^{p(k)}. \quad (5)$$

O bloco utiliza uma janela de cinco amostras $\{x_0, x_1, x_2, x_3, x_4\}$, sendo x_2 a amostra central associada à saída validada. As posições x_3 e x_4 preservam o lookahead de duas amostras herdado do contrato OpenDPD. O conjunto de termos possui três termos lineares, com $p = 0$, e 36 termos não lineares, com $p = 1, 2, 3, 4$. Para cada ordem não linear são usadas nove combinações de posição:

$$(m_x, m_a) \in \{(0, 0), (1, 1), (2, 2), (0, 1), (1, 2), (2, 3), (0, 2), (1, 3), (2, 4)\}. \quad (6)$$

A magnitude é calculada por raiz inteira:

$$|x_m| = \sqrt{x_{i,m}^2 + x_{q,m}^2}, \quad (7)$$

e as potências são mantidas em escala Q1.15. Para $p > 0$,

$$\phi_i = \frac{x_{i,m_x} |x_{m_a}|^p}{2^{15}}, \quad \phi_q = \frac{x_{q,m_x} |x_{m_a}|^p}{2^{15}}. \quad (8)$$

A multiplicação complexa de cada termo é:

$$\begin{aligned} y_i &\leftarrow y_i + \frac{\phi_i a_k - \phi_q b_k}{2^{16}}, \\ y_q &\leftarrow y_q + \frac{\phi_i b_k + \phi_q a_k}{2^{16}}. \end{aligned} \quad (9)$$

O deslocamento por 2^{16} corresponde à escala Q2.16 dos coeficientes. Após a soma dos 39 termos, a saída é saturada para Q1.15.

B. MACcore e Block-NLMS

O *MACcore* usa o feedback como base de regressão e a referência como alvo. Para cada amostra capturada,

$$\phi_k[n] = fb_{m_x(k)}[n] |fb_{m_a(k)}[n]|^{p(k)}. \quad (10)$$

O modelo corrente estima:

$$\hat{y}[n] = \sum_{k=0}^{38} w_k \phi_k[n], \quad (11)$$

e o erro de modelo é:

$$e[n] = ref[n] - \hat{y}[n]. \quad (12)$$

Para cada termo, o treinamento acumula:

$$N_k = \sum_n e[n] \phi_k^*[n], \quad (13)$$

$$D_k = \sum_n |\phi_k[n]|^2. \quad (14)$$

A atualização block-NLMS aplicada é:

$$w_k \leftarrow sat_{Q2.16} \left(w_k + \mu \frac{N_k}{\epsilon + D_k} \right). \quad (15)$$

No HDL atual, $\epsilon = 1$ e $\mu = 1/4$, implementado por deslocamento de dois bits. O treinamento executa até 20 épocas, mantendo o melhor conjunto de coeficientes pela norma L1 do erro:

$$E_{epoch} = \sum_n (|\Re\{e[n]\}| + |\Im\{e[n]\}|). \quad (16)$$

Após pelo menos 10 épocas, o treinamento pode parar antecipadamente caso o erro da época atual fique pior que o melhor erro observado.

C. MetricEngine

O *MetricEngine* calcula métricas leves, sintetizáveis e adequadas ao plano de controle. A magnitude L1 da saída DPD é:

$$mag_{dpd}[n] = |dpd_i[n]| + |dpd_q[n]|. \quad (17)$$

O erro REF-FB é:

$$err_{L1}[n] = |ref_i[n] - fb_i[n]| + |ref_q[n] - fb_q[n]|. \quad (18)$$

O drift entre esforço de predistorção e referência é:

$$drift_{L1}[n] = |mag_{dpd}[n] - (|ref_i[n]| + |ref_q[n]|)|. \quad (19)$$

As métricas são atualizadas por EWMA com fator $1/256$:

$$M \leftarrow M - \frac{M}{256} + v[n]. \quad (20)$$

Esses valores não são expressos em dB; em regime permanente, aproximam 256 vezes a média L1 do sinal observado. O pedido de retreinamento é gerado por:

$$\begin{aligned} retrain = & metrics_valid \wedge (metric_error > th_e \vee \\ & metric_drift > th_d \vee \\ & metric_clipping > th_c). \end{aligned} \quad (21)$$

VII. VALIDAÇÃO HDL

A validação foi conduzida no Questa, usando testbenches SystemVerilog. O fluxo foi construído de forma incremental: primeiro foram validados periféricos, bancos de coeficientes, captura e alinhamento; em seguida, os blocos críticos *GMPengine*, *MetricEngine* e *MACcore*; por fim, o fluxo completo em `dpd_top`.

O contrato OpenDPD-HDL foi validado por arquivos `.hex`. O banco `coef_bank_openDPD.hex` contém 78 palavras úteis, em formato real/imaginário, e margem até 128 posições. As amostras e saídas esperadas são armazenadas em Q1.15. O testbench do *GMPengine* carrega os coeficientes, aplica o dataset e compara a saída HDL com vetores calculados no mesmo modelo inteiro usado para validação. O teste integrado exercita a sequência:

```
bypass -> captura -> treino ->
coeficientes -> sync -> DPD ativo ->
métricas -> retreino.
```

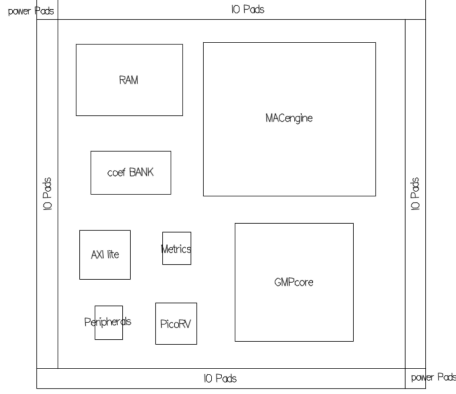
A validação das métricas permite alternar entre DPD desabilitado e DPD ativo. Em `bypass`, a saída observada se aproxima da referência e as métricas de drift tendem a valores baixos. Com DPD ativo, as métricas passam a observar o esforço de predistorção aplicado pelo *GMPengine*. Essa separação é importante porque o sinal DPD não deve ser idêntico à referência antes do PA; ele deve ser intencionalmente pre-distorcido para que a saída após o PA se aproxime do comportamento linear desejado.

VIII. AVALIAÇÃO FÍSICA EM SKY130

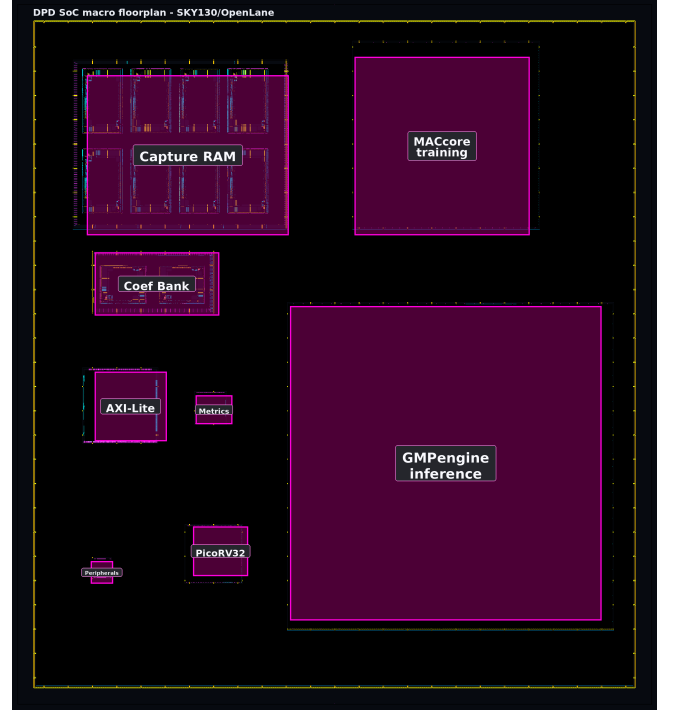
Para avaliação física, o projeto foi reorganizado em macros *OpenLane*. Blocos menores, como *PicoRV32*, *AXI-Lite*, periféricos e *MetricEngine*, foram sintetizados individualmente. A RAM de captura e os bancos de coeficientes foram tratados como macros com SRAM, evitando implementação de grandes memórias em flip-flops. Os blocos críticos *GMPengine* e *MACcore* receberam versões específicas para reduzir área e pressão de roteamento.

O *GMPengine* é o bloco de maior pressão temporal por pertencer ao caminho rápido. Na versão *OpenLane* atual, ele foi serializado em quatro fases, preservando 39 termos complexos, amostras Q1.15 e coeficientes Q2.16. Com essa arquitetura, o throughput é $F_{clk}/4$. O fechamento físico preliminar indicou cerca de 90,9 MHz, equivalente a 22,7 MS/s. Para sustentar 24 MS/s com margem, a próxima etapa precisa superar 96 MHz ou reduzir o intervalo de iniciação.

Os resultados da Tabela II devem ser interpretados como estimativas de engenharia para a próxima iteração física. O top atual é um scaffold macro-level, sem padframe definitivo e sem



(a) Floorplan manual preliminar.



(b) Floorplan macro-level gerado no OpenLane.

Figura 4: Evolução do floorplan físico: proposta manual e organização macro-level do top scaffold.

Tabela II: Resultados físicos preliminares em OpenLane/SKY130.

Bloco	Run	Status	Runtime	Área (mm ²)	Células	Cell/mm ²	Fmax (MHz)	DRC/LVS
picorv32	signoff_100m_01	signoff ok	0h11m28s	0.377	10114	26818.6	100.0	0/0
axi_ctrl	signoff_100m_01	signoff ok	0h03m30s	0.640	1713	2676.6	100.0	0/0
metric_engine	signoff_100m_01	signoff ok	0h03m28s	0.116	2593	22280.8	100.0	0/0
peripherals	signoff_100m_01	signoff ok	0h01m15s	0.044	755	17328.1	100.0	0/0
capture_ram	macro_route_100m_01	macro ok	0h04m02s	4.140	330	79.7	100.0	0/0
coef_bank	macro_route_100m_01	macro ok	0h01m25s	0.845	152	179.9	100.0	0/0
gmp_engine	signoff_100m_01	signoff ok	12h42m04s	12.206	328576	26919.2	90.9	0/0
mac_engine	signoff_100m_no_prefill_01	signoff ok	2h11m45s	3.987	91948	23062.2	100.0	0/0
soc_top_scaffold	tapeout_full_05	DRC ok, LVS aberto	1h34m10s	51.000	8	0.2	90.9	0/35

toda a conectividade funcional de chip. Portanto, embora o GDS tenha sido gerado e o DRC esteja limpo, o LVS aberto no top é esperado nessa etapa. As macros individuais, por outro lado, fornecem informações úteis sobre custo relativo dos blocos. O *GMPengine* domina área e tempo de processamento físico, enquanto o *MACcore*, mesmo maior que blocos de controle, permanece compatível com operação em background.

IX. DISCUSSÃO

Os resultados mostram que a maior dificuldade do projeto não está apenas no algoritmo, mas no contrato entre algoritmo, quantização e implementação física. O OpenDPD indicou que o GMP com 39 termos é adequado para o dataset estudado, mas a implementação fiel desse modelo em SKY130 exige multiplicações complexas, cálculo de magnitude, potências e acumulação de muitos termos. A serialização em quatro

fases reduziu área e permitiu avançar no fluxo OpenLane, mas aproximou o throughput do limite mínimo para 24 MS/s.

O *MetricEngine* ilustra uma decisão recorrente em hardware de telecomunicações: nem todas as métricas usadas em software devem ser reproduzidas em tempo real no chip. NMSE, EVM e ACLR continuam adequadas para análise offline e documentação, enquanto métricas L1/EWMA são mais apropriadas para supervisão embarcada. O *MACcore* segue a mesma filosofia: ele não implementa um LS/RLS matricial completo, mas uma atualização block-NLMS sintetizável que pode operar em janelas capturadas sem perturbar o caminho de transmissão.

X. CONCLUSÃO

Este trabalho consolidou uma cadeia de desenvolvimento para DPD aplicada a transmissores de televisão digital de próxima geração, conectando geração de dataset, treinamento

OpenDPD, implementação HDL e avaliação física preliminar em SKY130. O estudo definiu um contrato técnico claro: sinal I/Q em 24 MS/s, amostras Q1.15, coeficientes Q2.16, modelo GMP de 39 termos complexos, treinamento block-NLMS em background e métricas leves para supervisão.

A validação HDL confirmou a coerência funcional dos blocos críticos e do fluxo integrado de operação, incluindo bypass inicial, captura, treinamento, escrita de coeficientes, troca sincronizada de banco, inferência DPD e solicitação de retreinamento. A avaliação OpenLane mostrou que as macros menores e os blocos críticos podem ser levados ao fluxo físico, mas também evidenciou que o *GMPengine* é o principal gargalo de área, roteamento e temporização.

Como próximos passos, destacam-se o fechamento temporal do *GMPengine* acima de 96 MHz, a definição de um padframe realista, a integração funcional completa do top-level, a validação com feedback de PA medido ou modelado no testbench e a comparação com datasets mais longos e diferentes condições de amplificador. Esses passos são necessários para transformar o scaffold físico atual em uma implementação de chip fechada para fabricação.

REFERÊNCIAS

- [1] OpenROAD Project and Efabless, “OpenLane: Automated RTL to GDSII Flow,” GitHub repository, 2026.
- [2] SkyWater Technology and Google, *SkyWater SKY130 Open Source PDK*, SkyWater Technology and Google, 2026.
- [3] Y. Wu, G. D. Singh, M. Beikmirza, L. C. N. de Vreede, M. Alavi, and C. Gao, “OpenDPD: An Open-Source End-to-End Learning and Benchmarking Framework for Wideband Power Amplifier Modeling and Digital Pre-Distortion,” *arXiv preprint arXiv:2401.08318*, 2024.
- [4] TU Delft EMI Lab, “OpenDPD Repository,” GitHub repository, 2026.
- [5] GNU Radio Project, “GNU Radio: The Free and Open Software Radio Ecosystem,” Project website, 2026.
- [6] R. Economos, “gr-atsc3: ATSC 3.0 Transmitter for GNU Radio,” GitHub repository, 2026.
- [7] Forum SBTVD, *SBTVD TV 3.0 Operational Guide 01: Physical Layer*, Forum do Sistema Brasileiro de TV Digital Terrestre, 2026.
- [8] Forum SBTVD, “SBTVD TV 3.0 Phase 3 Physical Layer Lab Test Report,” Forum do Sistema Brasileiro de TV Digital Terrestre, Tech. Rep., 2023.
- [9] Forum SBTVD, “SBTVD TV 3.0 Phase 3 Physical Layer Field Test Report,” Forum do Sistema Brasileiro de TV Digital Terrestre, Tech. Rep., 2024.
- [10] ATSC, *A/300:2024-04, ATSC 3.0 System*, Advanced Television Systems Committee, 2024.
- [11] ATSC, *A/321:2024-04, System Discovery and Signaling*, Advanced Television Systems Committee, 2024.
- [12] ATSC, *A/322:2024-09, Physical Layer Protocol*, Advanced Television Systems Committee, 2024.
- [13] ATSC, *A/324:2024-04, Scheduler/Studio to Transmitter Link*, Advanced Television Systems Committee, 2024.
- [14] ABNT, *ABNT NBR 25601:2025, TV 3.0 – Camada física*, Associação Brasileira de Normas Técnicas, 2025.

[1] OpenROAD Project and Efabless, “OpenLane: Automated RTL to